Counterexamples to the Akbari–Cameron–Khosrovshahi row-space conjecture

Alex Chan* Weilin Meng† Kurt Sweat‡

Abstract

Three counterexamples to the Akbari–Cameron–Khosrovshahi row-space conjecture are constructed. All are connected reduced graphs of 52 vertices with adjacency rank 14 and diameter 3. The real row space of the adjacency matrix for each graph contains 53 Boolean vectors: the zero vector and the 52 adjacency vectors. Construction of the graphs is shown using four short integer lists. We verify the constructions using exact arithmetic on all 2^{14} Boolean vectors. Our construction employs the fixed-rank extension theorem of Akbari–Cameron–Khosrovshahi, and shows that a combination of two constraints are satisfied. Finally, we show that Boolean row-space intersections for graphs obtained from false-twin blowups are preserved. This gives a connected counterexample of order 52 and rank 14 with 53 Boolean row-space vectors.

Keywords. Adjacency matrix; graph rank; Boolean vector; row space; computer-assisted proof.
Mathematics Subject Classification (2020). 05C50, 15A03.

1 Introduction

The rows of a graph's adjacency matrix are Boolean vectors in the graph's real row space. Akbari, Cameron and Khosrovshahi asked if every graph with an edge has a different non-zero Boolean vector in the real row space. Here we construct three such examples. Each example has 52 vertices, rank 14 and diameter 3. The only non-zero Boolean vectors in the real row space of each example is the 52 adjacency vectors.

All graphs in this paper are finite, simple and undirected. The rank of a graph is the rank over $\mathbb{R}$ of its adjacency matrix. Let A be an $n \times n$ real matrix, we set

$$\mathcal{B}(A) = \mathrm{Row}_{\mathbb{R}}(A) \cap \{0,1\}^n, \qquad \mathcal{R}(A) = \{A_{i,*} : 0 \le i < n\},$$

where row spaces are represented as row vectors.

Every adjacency matrix satisfies

$$\{\mathbf{0}\} \cup \mathcal{R}(A) \subseteq \mathcal{B}(A). \tag{1}$$

The question is whether this inclusion is always strict when the graph contains an edge. This question comes from the manuscript of Akbari, Cameron and Khosrovshahi [1, Question 2]. The universally affirmative form of this question was presented as an open problem to the British Combinatorial Conference [4] and it continues to be of interest (see author's webpage [5]).

*Harvard University and NBER. achan@hbs.edu. Funded by Harvard Business School as a subproject within Project STANF..

†Harvard Business School.

‡UT Southwestern Medical Center.

Conjecture 1.1 (ACK row-space conjecture)**.** If A is the adjacency matrix of a graph with at least one edge, then

$$\mathcal{B}(A) \setminus (\{\mathbf{0}\} \cup \mathcal{R}(A)) \neq \varnothing.$$

A graph that satisfies this statement is called an ACK graph. A counterexample to this statement demonstrates equality in (1). For the remainder of this paper, when we say a graph is reduced, we mean it has no isolated vertices, no two vertices with the same open neighborhood and it has at least one edge. In a reduced graph of order n, the set on the left hand side of (1) has $n+1$ elements.

Several classes of graphs satisfy the conjecture. For one, a nonsingular adjacency matrix has row space $\mathbb{R}^n$, and, for a regular graph with an edge, the all-ones vector is in the row space. Jobson [6] looked at affirmative graph families and related matrix questions. He also gave counterexamples to a generalization that allows asymmetric Boolean matrices. Bera [3] proved the conjecture for connected graphs of diameter at least four, and got some other affirmative cases of diameter two and three. Sciriha, Kaur, Borg, and Debono [7] gave some structural restrictions on potential counterexamples. Bera also showed how some affirmative graphs families can be constructed from given graphs by using certain operations. Other affirmative families and operations on graphs were shown by Akansha and Sivakumar [2], and they still stated the conjecture.

Our main result is the following.

Theorem 1.2. *There are three pairwise nonisomorphic connected reduced graphs $G_{550}, G_{563}, G_{570}$ on 52 vertices, each of rank 14 and diameter 3, such that*

$$\mathcal{B}(A_G) = \{\mathbf{0}\} \cup \mathcal{R}(A_G). \tag{2}$$

Their numbers of edges are 550, 563, and 570, respectively. In particular, Conjecture 1.1 is false.

We prove the main result in two parts. In the first, we state the connected graphs of rank 14 and diameter 3 by using a common nonsingular principal block of order 14, and by listing three extension masks. In the second, we check all possible Boolean vectors on the first 14 coordinates and show that exactly 53 of the prefixes extend to Boolean row-space vectors. These are the all-zero vector and the 52 adjacency row prefixes. The construction and complete verification are given. Therefore, no search seed is required, nor access to a private graph repository or other graph databases.

In this section, we define the criterion for our search. Then we specify the characteristic vectors of our examples, without claiming that our examples satisfy the criterion for minimum order or for minimum rank. The result does not address a separate conjecture about the maximum order of a reduced graph of a given rank. Finally, we give an additional implementation of our result in Lean in Section 6, and present its specification and the correspondence between the data stored in the implementation and the matrices of this paper. The proof of Theorem 1.2 relies neither on the specification nor on the implementation of the additional Lean result.

Corollary 1.3. *For every integer $N \geq 52$, there is a connected graph H_N of order N, rank 14, and diameter 3 such that*

$$\mathcal{B}(A_{H_N}) = \{\mathbf{0}\} \cup \mathcal{R}(A_{H_N}), \qquad |\mathcal{B}(A_{H_N})| = 53.$$

The larger graphs have repeated adjacency rows. We prove this corollary by false-twin blowups in Section 5.

2 Fixed-rank extensions and the search criterion

We start by explaining how the rank-determining principal block defines the row space. Next, we define the conditions to eliminate all remaining Boolean vectors.

Let $r \geq 2$, and let B be a symmetric nonsingular Boolean matrix of order r with zero diagonal. Let W be an $n \times r$ Boolean matrix whose first r rows form B, and set

$$A = WB^{-1}W^{\mathsf{T}}, \qquad R = B^{-1}W^{\mathsf{T}}. \tag{3}$$

The requirements that A be Boolean and have zero diagonal will be imposed separately. Writing $W = \binom{B}{U}$ gives

$$A = \begin{pmatrix} B & U^{\mathsf{T}} \\ U & UB^{-1}U^{\mathsf{T}} \end{pmatrix}, \qquad R = (I_r \;\; B^{-1}U^{\mathsf{T}}). \tag{4}$$

Lemma 2.1. *The matrix A has rank r. The map*

$$f : \mathbb{R}^r \longrightarrow \mathrm{Row}_{\mathbb{R}}(A), \qquad f(x) = xR,$$

is a linear isomorphism whose inverse takes the first r coordinates. Consequently,

$$\mathcal{B}(A) = \{xR : x \in \{0,1\}^r,\ xR \in \{0,1\}^n\}. \tag{5}$$

The rows of W map to the corresponding rows of A.

Proof. The identity $A = WR$ gives $\mathrm{Row}_{\mathbb{R}}(A) \subseteq \mathrm{Row}_{\mathbb{R}}(R)$. Conversely, $R = B^{-1}A_{\{0,\ldots,r-1\},*}$ gives the reverse inclusion. The first r columns of R form I_r, so R has rank r and xR has prefix x. The last assertion is again $A = WR$. □

For Boolean row vectors x, y of length r, define

$$\beta(x,y) = xB^{-1}y^{\mathsf{T}}, \qquad q(x) = \beta(x,x).$$

Let $b_0, \ldots, b_{r-1}$ be the rows of B, and put

$$P_B = \{c \in \{0,1\}^r : q(c) = 0\} \setminus \{\mathbf{0}, b_0, \ldots, b_{r-1}\}.$$

The *compatibility graph* Γ_B has vertex set P_B; distinct c, d are adjacent when $\beta(c,d) \in \{0,1\}$. This is the construction of [1, Section 3], expressed in Boolean neighborhood coordinates.

Given a clique $C \subseteq P_B$, let W_C have rows $b_0, \ldots, b_{r-1}$ followed by the members of C, and set $A_C = W_C B^{-1} W_C^{\mathsf{T}}$. This matrix is symmetric and has zero diagonal. Its entries are Boolean because $\beta(b_i, c) = c_i$ and the other pairings are controlled by the clique condition. Its rows are distinct and nonzero, as their prefixes are the distinct nonzero rows of W_C.

Maximality controls the isotropic prefixes. To exclude all additional Boolean prefixes, we also need to control those with $q(x) \neq 0$. For each such $x \in \{0,1\}^r$, define

$$E_x = \{c \in P_B : \beta(x,c) \notin \{0,1\}\}. \tag{6}$$

Proposition 2.2. *For a clique C in Γ_B, the equality $\mathcal{B}(A_C) = \{\mathbf{0}\} \cup \mathcal{R}(A_C)$ holds if and only if:*

1. *C is an inclusion-maximal clique of Γ_B;*
2. *$C \cap E_x \neq \varnothing$ for every $x \in \{0,1\}^r$ with $q(x) \neq 0$.*

Proof. By Lemma 2.1, a Boolean prefix x gives a Boolean row-space vector precisely when $\beta(x,c) \in \{0,1\}$ for every $c \in C$. Zero, the base rows, and members of C always satisfy these tests.

If $q(x) = 0$ and x is not one of those prescribed prefixes, then $x \in P_B \setminus C$. It survives all extension coordinates exactly when it is adjacent to every member of C. Excluding all such x is equivalent to inclusion-maximality.

If $q(x) \neq 0$, then x cannot be a row of W_C, all of whose rows are isotropic. It is excluded exactly when some member of C lies in E_x. These cases exhaust $\{0,1\}^r$. □

The distinction matters: maximality in the fixed-rank compatibility search excludes additional isotropic prefixes but not nonisotropic Boolean prefixes. For $B = A(K_2)$, the compatibility graph has no vertices, but $q(1,1) = 2$ and the row space still contains $(1,1)$.

Corollary 2.3. *If a clique C of Γ_B intersects every E_x with $q(x) \neq 0$, then any inclusion-maximal clique containing C yields an ACK counterexample. If one such E_x is empty, no clique in Γ_B yields a counterexample.*

Proof. Extending a clique preserves every nonempty intersection with E_x. Apply Proposition 2.2 after maximal completion. The second claim follows from the necessary covering condition. □

We can therefore search for a clique satisfying the covering constraints and then extend it to a maximal clique. This reduction explains the construction. To verify the final graphs, we instead check every Boolean prefix directly.

3 The three graphs

We now give the complete data for the three examples. We encode each Boolean row by an integer, with the least significant bit representing coordinate 0.

For $0 \leq s < 2^{14}$, write

$$\epsilon(s) = (\epsilon_0(s), \ldots, \epsilon_{13}(s)), \qquad s = \sum_{j=0}^{13} 2^j \epsilon_j(s),$$

with $\epsilon_j(s) \in \{0,1\}$. The least significant bit is coordinate 0. The common principal block B has the ordered row masks

$$\begin{aligned} b = (&4018, 11465, 5136, 6034, 8429, 721, 3634, 7995,\\ &14985, 1513, 6863, 13763, 3468, 2322). \end{aligned} \tag{7}$$

Thus $B_{ij} = \epsilon_j(b_i)$. For each ordered list c_m below, let W_m have row masks (b, c_m) and define

$$A_m = W_m B^{-1} W_m^{\mathsf{T}}, \qquad m \in \{550, 563, 570\}. \tag{8}$$

The extension list c_{550}.

```
384, 676, 1497, 1517, 1810, 2031, 2444, 2692,
2859, 2998, 3080, 3362, 3464, 4289, 4559, 4803,
5378, 5520, 5650, 5715, 6099, 6162, 6227, 6284,
6349, 6566, 8385, 9232, 9248, 9904, 10761, 12091,
13379, 13807, 14035, 14051, 14635, 15410
```

The extension list c_{563}.

```
384, 449, 793, 1129, 1296, 1452, 1810, 2031,
2060, 2692, 3080, 3390, 3451, 4875, 5378, 5520,
6099, 6566, 6611, 6665, 6838, 7586, 8385, 8745,
9232, 9616, 12739, 13698, 13807, 13881, 14107, 14123,
14563, 15488, 963, 5650, 6349, 10445
```

The extension list c_{570}.

```
89, 384, 493, 963, 1497, 2617, 2843, 3120,
3346, 3362, 3712, 4112, 5152, 5241, 5467, 6099,
6182, 6271, 6284, 8385, 8457, 9011, 9089, 9248,
9632, 10278, 10445, 10662, 12169, 12355, 13011, 13537,
13609, 13807, 14051, 14883, 14989, 15488
```

These masks and (8) specify each graph without a search seed or external database. Their invariants appear in Table 1. All three lie in the diameter-three case not settled by the general diameter result of [3].

Table 1: Invariants of the three graphs. The Boolean count includes zero.

Graph	Order	Edges	Rank	Diameter	Degree range	$\lvert\mathcal{B}(A)\rvert$
G_{550}	52	550	14	3	6–32	53
G_{563}	52	563	14	3	9–32	53
G_{570}	52	570	14	3	11–32	53

4 Exact certificates and proof of the main theorem

We verify the examples by reducing each row-space calculation to an integer calculation on 14 coordinates. From the displayed masks, we recover the reconstruction matrices

$$D_m = 3B^{-1}W_m^{\mathsf{T}} \in \mathbb{Z}^{14\times 52}. \tag{9}$$

For a prefix mask s, define

$$h_m(s,j) = \sum_{k=0}^{13} \epsilon_k(s)(D_m)_{kj} \quad (0 \le j < 52), \tag{10}$$

and let M_m be the set of the 52 masks occurring in (b, c_m).

Lemma 4.1 (Finite certificate)**.** *The matrix B is symmetric, has zero diagonal, and has determinant -9. For each $m \in \{550, 563, 570\}$, the following hold.*

1. *The masks in M_m are distinct and nonzero. The matrix A_m is Boolean and has zero diagonal.*
2. *The matrix D_m is integral and satisfies*

$$3(A_m)_{ij} = \sum_{k=0}^{13} (A_m)_{ik}(D_m)_{kj} \quad (0 \le i, j < 52). \tag{11}$$

3. *For $0 \le s < 2^{14}$,*

$$\big(h_m(s,j) \in \{0,3\} \text{ for every } 0 \le j < 52\big) \quad \Longleftrightarrow \quad s \in \{0\} \cup M_m. \tag{12}$$

4. *The graph with adjacency matrix A_m is connected and has the edge count, diameter, and degree range in Table 1.*

Exact verification. The matrix in (7) has a determinant of -9 after using rational Gaussian elimination. Using (8) and (9), we obtain W_m, D_m, A_m. We verify the values in the diagonal, the mask values, and all 2704 identities in (11). We evaluate (10) for the 16384 masks. Exactly 53 masks are accepted and the set is $0 \cup M_m$. The other 16331 masks are rejected. We obtain the graph invariants from degree sums and do a breadth-first search from each vertex. The checking algorithm is in Appendix A. It is written using only displayed integers, Boolean logic, and basic arithmetic. We choose a subset of $16384 - 53 = 16331$ prefixes and reject them.

For comparison with Proposition 2.2, the common block has 741 isotropic Boolean vectors including zero and the base rows. Thus $|P_B| = 726$ and there are $15,643$ non-isotropic prefixes. Each displayed set of 38 is a clique and rejects its 688 unselected candidate extensions, as well as the $15,643$ non-isotropic prefixes. The split is shown in Table 2 and is verified by the companion code. □

Table 2: Prefix classification for each graph.

Prefix class	Number
Zero and base rows	15
Selected extension rows	38
Unselected isotropic extensions (all rejected)	688
Nonisotropic prefixes (all rejected)	$15,643$
Total	$16,384$

Proof of Theorem 1.2. Lemma 4.1 makes A_m a symmetric Boolean matrix with zero diagonal, hence the adjacency matrix of a simple graph. Its first 14 rows and columns form B. Lemma 2.1 gives rank 14 and expresses each row-space vector uniquely as $f(x) = xB^{-1}W_m^{\mathsf{T}}$ with prefix $x \in \mathbb{R}^{14}$.

If $f(x)$ is Boolean, so is x. Write $x = \epsilon(s)$. Equations (9) and (10) give $3f(x)_j = h_m(s, j)$ for every j. By (12), either $s = 0$ or x is a row of W_m. Lemma 2.1 identifies $f(x)$ as zero or the corresponding adjacency row. The reverse inclusion is (1).

Distinct nonzero prefixes imply distinct nonzero adjacency rows, so the graphs are reduced and their Boolean intersections have 53 elements. Connectivity and diameter follow from Lemma 4.1; different edge counts prove pairwise nonisomorphism. □

5 False-twin blowups

We now show that duplicating vertices as false twins preserves the entire Boolean row space intersection. False-twin blowups are standard in the study of graph rank. See [1, Section 2] for more details. We include a full proof, since in addition to preserving the rank of the

Boolean matrices, we need to preserve the Boolean vectors as well.

Proposition 5.1. *Let H be obtained from G by replacing each vertex i with a nonempty independent set V_i, with all edges between V_i and V_j when $ij \in E(G)$. There is a bijection*

$$\mathcal{B}(A_G) \longrightarrow \mathcal{B}(A_H)$$

that sends zero to zero and the set of distinct adjacency rows of G onto the set of distinct adjacency rows of H. In particular, G is an ACK graph if and only if H is an ACK graph, and their adjacency matrices have the same rank.

Proof. Put $A = A_G$, $A' = A_H$, and let T be the $|V(H)| \times |V(G)|$ membership matrix, with $T_{vi} = 1$ if $v \in V_i$. Then $A' = TAT^{\mathsf{T}}$. Choosing one representative of each class gives a left inverse L with $LT = I$. The identities

$$A' = T(AT^{\mathsf{T}}), \qquad AT^{\mathsf{T}} = LA'$$

imply

$$\mathrm{Row}_{\mathbb{R}}(A') = \{xT^{\mathsf{T}} : x \in \mathrm{Row}_{\mathbb{R}}(A)\}. \tag{13}$$

The map $x \mapsto xT^{\mathsf{T}}$ repeats coordinate x_i on V_i. It is injective, preserves nonzero vectors, and has Boolean output exactly when its input is Boolean. The row of A' at any vertex in V_i is $A_{i,*}T^{\mathsf{T}}$. These observations prove both bijections. Injectivity on the row space also proves rank preservation. □

Proof of Corollary 1.3. Start with a graph in Theorem 1.2. Replace one vertex by an independent set of size $N - 51$ and leave the other classes singleton. Proposition 5.1 proves the rank and Boolean-intersection claims. The graph remains connected. Distances between different classes are unchanged, while distinct vertices in one class have distance two. The diameter therefore remains three. □

Isolated vertices are neutral, since

$$\mathrm{Row}_{\mathbb{R}}\begin{pmatrix} A & 0 \\ 0 & 0 \end{pmatrix} = \mathrm{Row}_{\mathbb{R}}(A) \times \{\mathbf{0}\}.$$

If a graph has two components each containing an edge, the sum of a nonzero adjacency row from each component is a new Boolean row-space vector. Hence a counterexample consists of one connected nontrivial counterexample and possibly isolated vertices. Together with Proposition 5.1, this justifies searching connected reduced graphs without asserting that every counterexample must itself be connected or reduced.

6 Formal specification and witness encoding

The exact verification in Section 4 is reproducible from the data and checker supplied with this paper. We describe the additional Lean implementation separately. In particular, a record of a successful build does not replace the complete sources needed to reproduce that build.

6.1 The statement being certified

Appendix B gives definitions for `IsBoolean`, `InRealRowSpace`, `IsSimpleAdjacency`, `Conjecture`, and `IsCounterexample`. The definition for the row space quantifies over real coefficients. Simple adjacency requires zero diagonal and symmetry and Boolean entries. The conjecture requires an edge and asks for a Boolean row space vector that is nonzero and different from all adjacency rows. These are the requirements for Conjecture 1.1.

The final counterexample declarations have the types

```
ACK.Witness550.isCounterexample :
  ACK.IsCounterexample ACK.Witness550.adjacency
ACK.Witness563.isCounterexample :
  ACK.IsCounterexample ACK.Witness563.adjacency
ACK.Witness570.isCounterexample :
  ACK.IsCounterexample ACK.Witness570.adjacency
```

Each namespace also contains `conjecture_false`, of type ¬ `ACK.Conjecture`. The counterexample predicate says that every Boolean vector in the real row space is zero or an adjacency row. Given this predicate and the existence of an edge, the negation of the conjecture follows directly.

6.2 Identifying the input graphs

We make the input convention explicit. For the matrix A_m defined in (8), put

$$\mu_m(i) = \sum_{j=0}^{51} 2^j (A_m)_{ij}, \qquad 0 \le i < 52. \tag{14}$$

Since A_m is Boolean, $\mu_m(i)$ is an integer between 0 and $2^{52} - 1$. These are full adjacency-row masks, whereas the masks in Section 3 encode only the first 14 coordinates. The Lean witnesses store a list `rowMasks` and use the convention

```
def edge (i j : Fin 52) : Bool :=
  (rowMasks.getD i.val 0).testBit j.val

def adjacency : Matrix (Fin 52) (Fin 52) Real :=
  fun i j => if edge i j then 1 else 0
```

Lemma 6.1. *Suppose that a stored list has length* 52 *and its entry at index* i *is* $\mu_m(i)$ *for every* $0 \le i < 52$. *Then its real adjacency matrix under the displayed convention is* A_m *entry by entry.*

Proof. By Lemma 4.1, each $(A_m)_{ij}$ is 0 or 1. Uniqueness of binary expansion in (14) therefore identifies bit j of $\mu_m(i)$ with $(A_m)_{ij}$. Reading that bit and interpreting it as a real entry gives the claim. □

The supplement contains adjacency-row-masks.json computed from (8). If supplied with the three Lean witness files, the program audit_lean_inputs.py does mask checking for each of the 52 stored masks for each graph against these masks. It uses the same vertex ordering as Section 3. This gives a way to audit the input correspondence. This package doesn't contain the private Lean source files, so it can't verify that correspondence. The utility compares exact input data and does not prove Lemma 6.1 in Lean.

6.3 Certificate checking and source availability

The Lean proof first checks (11). Using a linearity lemma, reconstruction is then extended to all real row space vectors. A certificate records one of three things for each Boolean prefix: zero, an existing row, or a coordinate whose integer numerator is neither 0 nor 3. The finite check covers all 16,384 prefixes; a structural binary-encoding proof shows that all Boolean prefixes are represented. Completing reconstruction uniqueness gives the final classification over $\mathbb{R}$.

It is reported that the recent builds use Lean's ordinary decision procedure. The axiom reports for the three counterexample theorems and the three refutation theorems list only `propext`, `Classical.choice` and `Quot.sound`. They do not list `sorryAx` or `Lean.ofReduceBool`. The rank, diameter, degree and edge counts, Proposition 2.2, and the blowup argument are proved or checked separately in this paper and are not separately formalized in that development.

The source repository is

https://github.com/alexgocardinal/ack-rowspace-counterexample.

The reported proof revision is `5b04adb`. It uses Lean 4.35.0 and Mathlib revision

56a005855ea17e0b0958f73ec801ae140b0a1679.

With the complete project, the reproduction commands are

```
lake exe cache get
lake build
lake env lean Verification.lean
python3 audit_lean_inputs.py /path/to/lean-project
```

The repository is private at the time of this writing. The current review package includes the complete, exact checker, graph data, formal specification and input comparison utility. It does not, however, include a frozen copy of the full Lean project. As such, independent replication of the other Lean certificates requires access to the source, including the certificate files, toolchain, dependency manifest, and verification entrypoint. Neither the reported Lean build nor input comparison is used as a premise of the exact proof in Section 4.

A An elementary exact checker

The code below checks determinants, the conditions on simple graphs, the reconstruction identities, the entire Boolean classification, and the graph invariants in Table 1. Its inputs are the four lists in Section 3. The accompanying verify$_a$*ck.pyincludesthoselistsandisrunnablewithstandard*`verify_ack.py`*.Allcalculatio*

```
from fractions import Fraction as F
from collections import deque

if not __debug__:
    raise RuntimeError("Run this checker without Python optimization (-O).")

def inverse_and_det(B):
    r = len(B)
    X = [[F(v) for v in row] + [F(i == j) for j in range(r)]
         for i, row in enumerate(B)]
    det = F(1)
    for k in range(r):
        p = next(i for i in range(k, r) if X[i][k])
        if p != k:
            X[k], X[p] = X[p], X[k]
            det = -det
        a = X[k][k]
        det *= a
        X[k] = [v/a for v in X[k]]
        for i in range(r):
            if i != k:
                a = X[i][k]
                X[i] = [v-a*w for v, w in zip(X[i], X[k])]
    return [row[r:] for row in X], det

def verify(base, extra, expected_edges, degree_range):
    r, n = 14, 52
    masks = base + extra
    assert len(base) == r and len(masks) == n
    assert len(set(masks)) == n and 0 not in masks
    assert all(0 <= s < 2**r for s in masks)
    W = [[(s >> k) & 1 for k in range(r)] for s in masks]
    B = W[:r]
    assert all(B[i][j] == B[j][i]
```

```
          for i in range(r) for j in range(r))
assert all(B[i][i] == 0 for i in range(r))
inv, det = inverse_and_det(B)
assert det == -9
R = [[sum(inv[k][l]*W[j][l] for l in range(r))
      for j in range(n)] for k in range(r)]
assert all((3*v).denominator == 1 for row in R for v in row)
D = [[int(3*v) for v in row] for row in R]
A = [[sum(W[i][k]*R[k][j] for k in range(r))
      for j in range(n)] for i in range(n)]
assert all(v in (0, 1) for row in A for v in row)
A = [[int(v) for v in row] for row in A]
assert all(A[i][i] == 0 for i in range(n))
assert all(A[i][j] == A[j][i]
          for i in range(n) for j in range(n))
assert [row[:r] for row in A] == W
assert all(3*A[i][j] == sum(A[i][k]*D[k][j] for k in range(r))
          for i in range(n) for j in range(n))
assert len(set(map(tuple, A))) == n

# Gray-code enumeration: each update changes exactly one bit.
assert all((9*v).denominator == 1 for row in inv for v in row)
Q = [[int(9*v) for v in row] for row in inv]
values, y, q, previous = [0]*n, [0]*r, 0, 0
accepted, isotropic, nonisotropic = set(), 0, 0
for t in range(2**r):
    s = t ^ (t >> 1)
    if t:
        changed = s ^ previous
        k = changed.bit_length() - 1
        sign = 1 if s & changed else -1
        values = [v+sign*d for v, d in zip(values, D[k])]
        q += 2*sign*y[k] + Q[k][k]
        y = [v+sign*Q[i][k] for i, v in enumerate(y)]
    previous = s
    isotropic += (q == 0)
    nonisotropic += (q != 0)
    if all(v in (0, 3) for v in values):
        accepted.add(s)
assert accepted == {0, *masks}
assert len(accepted) == 53
assert (isotropic, nonisotropic) == (741, 15643)

degrees = [sum(row) for row in A]
assert all(degrees) and sum(degrees) == 2*expected_edges
assert (min(degrees), max(degrees)) == degree_range
diameter = 0
for start in range(n):
    dist, queue = [-1]*n, deque([start])
    dist[start] = 0
    while queue:
        u = queue.popleft()
        for v in range(n):
            if A[u][v] and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    assert min(dist) == 0
    diameter = max(diameter, max(dist))
assert diameter == 3
```

```
    return {"order": n, "edges": expected_edges, "rank": r,
            "diameter": diameter, "degree_range": degree_range,
            "boolean_vectors": len(accepted), "accepted_masks": sorted(accepted),
            "isotropic_masks": isotropic, "nonisotropic_masks": nonisotropic,
            "rejected_masks": 2**r-len(accepted)}
```

B The formal specification

We display the predicates used in the reported Lean development. The scalar field is $\mathbb{R}$ throughout. The definitions below are in namespace `ACK`, with `Mathlib` imported and `BigOperators` open. We reproduce the specification for inspection; this listing alone is not a reproduction of the formal proof or its certificate checks.

```
def IsBoolean {n : ℕ} (x : Fin n → ℝ) : Prop :=
  ∀ j, x j = 0 ∨ x j = 1

def InRealRowSpace {n : ℕ}
    (A : Matrix (Fin n) (Fin n) ℝ)
    (x : Fin n → ℝ) : Prop :=
  ∃ coefficients : Fin n → ℝ,
    ∀ j, x j = ∑ i, coefficients i * A i j

def IsSimpleAdjacency {n : ℕ}
    (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  (∀ i, A i i = 0) ∧
  (∀ i j, A i j = A j i) ∧
  (∀ i j, A i j = 0 ∨ A i j = 1)

def Conjecture : Prop :=
  ∀ (n : ℕ) (A : Matrix (Fin n) (Fin n) ℝ),
    IsSimpleAdjacency A →
    (∃ i j, A i j = 1) →
    ∃ x : Fin n → ℝ,
      IsBoolean x ∧
      x ≠ 0 ∧
      InRealRowSpace A x ∧
      (∀ i, x ≠ A i)

def IsCounterexample {n : ℕ}
    (A : Matrix (Fin n) (Fin n) ℝ) : Prop :=
  IsSimpleAdjacency A ∧
  (∃ i j, A i j = 1) ∧
  (∀ x : Fin n → ℝ,
    IsBoolean x →
    InRealRowSpace A x →
    x = 0 ∨ ∃ i, x = A i)
```

References

[1] S. Akbari, P. J. Cameron, and G. B. Khosrovshahi, *Ranks and signatures of adjacency matrices*, unpublished manuscript (2004), Question 2. https://webspace.maths.qmul.ac.uk/p.j.cameron/preprints/ranksign.pdf.

[2] S. Akansha and K. C. Sivakumar, *Affirmative results on a conjecture on the column space of the adjacency matrix*, arXiv:2601.02746v1 (2026). https://arxiv.org/abs/2601.02746v1.

[3] S. Bera, Existence of a non-zero $(0, 1)$-vector in the row space of adjacency matrices of simple graphs, *Bull. Malays. Math. Sci. Soc.* **48** (2025), article 56. https://doi.org/10.1007/s40840-025-01838-0.

[4] P. J. Cameron (ed.), Research problems from the BCC21, *Discrete Math.* **310** (2010), 347–354. https://doi.org/10.1016/j.disc.2009.04.016.

[5] P. J. Cameron, *Peter Cameron's BCC problems*, problem BCC21.2, 12 September 2007. https://cameroncounts.github.io/web/QM/pjcatbcc.html.

[6] A. S. Jobson, *Avoiding, pretending, and querying: three combinatorial problems*, Ph.D. dissertation, University of Louisville, 2011. https://doi.org/10.18297/etd/693.

[7] I. Sciriha, B. Kaur, J. L. Borg, and M. Debono, Potential counter-examples to a conjecture on the column space of the adjacency matrix, *Special Matrices* **13** (2025), article 20250033. https://doi.org/10.1515/spma-2025-0033.